Development of a Multi-Format DVB-S2 Receiver Output Stream Analyzer with Anomaly Detection and Diagnostic Reporting

Y Vengala Rao
Roll No: 20231COM0001
Dept. of Computer Science & Eng.
Presidency University, India

Sk Samad
Roll No: 20231COM0031
Dept. of Computer Science & Eng.
Presidency University, India

C Rakeshwar
Roll No: 20231COM0008
Dept. of Computer Science & Eng.
Presidency University, India

***Abstract*—**Satellite communication networks rely on DVB-S2 to carry broadband data, cellular backhaul, and high-definition video over geostationary and non-geostationary links. In physical ground stations, receiver demodulators hand off decoded bitstreams across three fundamentally incompatible container formats: MPEG-2 Transport Streams (MPEG-TS), Generic Stream Encapsulation (GSE), and native DVB-S2 Baseband Frames (BBFrames). Diagnosing faults across these disparate streams is frustratingly difficult. Each format uses different packet boundaries, synchronization schemes, and error markers, while conventional network analyzers and broadcast tools generally support only one format at a time. In this paper, we present PRJ_111, a self-contained software suite for multi-format DVB-S2 receiver stream analysis. PRJ_111 implements an end-to-end analytical pipeline featuring deterministic health scoring based on ETSI TR 101 290 Priority-1 checks, unsupervised anomaly detection using format-specific Isolation Forest models, Shannon entropy profiling of multiplex streams, byte-accurate activity timelines, bounded Z-score explanations, semantically guarded cross-format comparisons, and automated multi-format reporting. We validated the system against real-world broadcast captures. On nominal streams, it confirmed 100.0% framing integrity; on corrupted captures, it pinpointed bit-level transport errors and container boundary truncations. A built-in regression suite of 304 automated tests (268 backend tests and 36 frontend integration tests) verifies the implementation with zero failures and zero regressions.

Index Terms*—Anomaly detection, Baseband frame, Digital video broadcasting, DVB-S2, Feature extraction, Generic stream encapsulation, Isolation Forest, MPEG transport stream, Receiver output analysis, Stream analysis.*

I. Introduction

Satellite links running DVB-S2 [1] handle massive volumes of broadcast video, broadband Internet traffic, and corporate data every day. When a satellite dish locks onto a downlink carrier, the receiver's front-end downconverts the signal to an intermediate frequency. Demodulator circuitry then performs carrier recovery, matched filtering, and dual-stage forward error correction (FEC) using inner LDPC and outer BCH codes [2]. Once error correction is complete, the receiver delivers a raw digital stream to host processing software or demultiplexers.

In real-world ground stations and teleports, however, this post-demodulator output does not come in a single neat format. Depending on how the transponder was provisioned, operators receive one of three distinct framing layers:

- **MPEG-2 Transport Stream (MPEG-TS):** The long-standing broadcast standard [3], packaging data into rigid 188-byte packets synchronized by `0x47` sync octets, with 13-bit Program Identifiers (PIDs) and 4-bit continuity counters.
- **Generic Stream Encapsulation (GSE):** The modern IP encapsulation protocol [4], which wraps variable-length network datagrams (IPv4, IPv6) with 2-bit fragmentation headers and CRC-32 checksums, avoiding MPEG-TS overhead.
- **DVB-S2 Baseband Frames (BBFrame):** The direct physical-link container [1], containing an 80-bit baseband header, Mode Adaptation flags (MATYPE), a Data Field Length (DFL) indicator, and an 8-bit CRC.

Troubleshooting these streams during station commissioning or downlink outages is often a painful process [5]. General-purpose packet sniffers like Wireshark expect Ethernet frames and stumble on raw, unencapsulated transport streams [6]. On the other hand, commercial broadcast monitors parse MPEG-TS well, but ignore GSE packets and baseband frames altogether [5].

Furthermore, field engineers face three recurring traps when analyzing captured streams:

1. **Brittle Parsers and Misidentified Files:** Raw captures arrive as unlabeled binary files (`.ts`, `.bin`, `.pcap`). Simple file-extension checks frequently guess wrong, and fragile parsers crash whenever they hit a corrupted packet or lost sync byte.
2. **Comparing Apples to Oranges:** Naively comparing packet counts between 188-byte MPEG-TS packets and 7-kilobyte Baseband Frames leads to meaningless conclusions, confusing operators during multi-feed audits.
3. **Speculative Root Causes and Fake Timestamps:** Many commercial tools invent smooth Megabit-per-second rate charts or guess at physical-layer issues (like blaming rain fade for packet drops) without access to actual receiver clock signals or signal-to-noise ratios.

To fix these problems, we created **PRJ_111**, a lightweight, standalone software workstation for analyzing DVB-S2 receiver output streams. PRJ_111 features a seven-stage analytical pipeline (F1–F7) that automatically detects stream formats from raw binary headers, parses units deterministically, calculates stream health (F1), discovers structural anomalies with an unsupervised Isolation Forest (F2), analyzes multiplex entropy (F3), renders byte-indexed activity timelines (F4), explains anomaly drivers with clamped Z-scores (F5), protects cross-stream comparisons with a strict semantic shield (F6), and exports comprehensive diagnostic reports (F7).

II. Related Work

A. Satellite Framing Standards and Protocols

The DVB-S2 standard (ETSI EN 302 307-1) defines framing, error correction, and modulation for digital satellite communication [1]. Morello and Mignone [2] broke down its physical architecture, showing how user packets get packed into baseband blocks protected by concatenated LDPC and BCH codes. Casini et al. [7] analyzed modulation choices up to 32-APSK, detailing the 80-bit Baseband Header (BBHeader), square-root raised cosine roll-off choices ($\alpha \in \{0.20, 0.25, 0.35\}$), and Data Field Length (DFL) bit budgets. Sklar [8] established the theoretical limits governing modulation capacity and framing overhead.

For network traffic, Fairhurst and Collini-Nocker [9] developed Generic Stream Encapsulation (ETSI TS 102 606-1) [4] as a lighter alternative to Multi-Protocol Encapsulation (MPE) and Unidirectional Lightweight Encapsulation (ULE, RFC 4326) [10]. GSE wraps IP datagrams into variable-length protocol data units with 32-bit CRC protection, ditching the fixed 188-byte MPEG framing overhead. Broadcast video, however, still runs on MPEG-2 Transport Streams (ISO/IEC 13818-1) [3]. ETSI TR 101 290 [11] sets the gold standard for measuring broadcast integrity, defining Priority-1 faults like lost synchronization, asserted Transport Error Indicator (TEI) flags, and Continuity Counter (CC) sequence jumps.

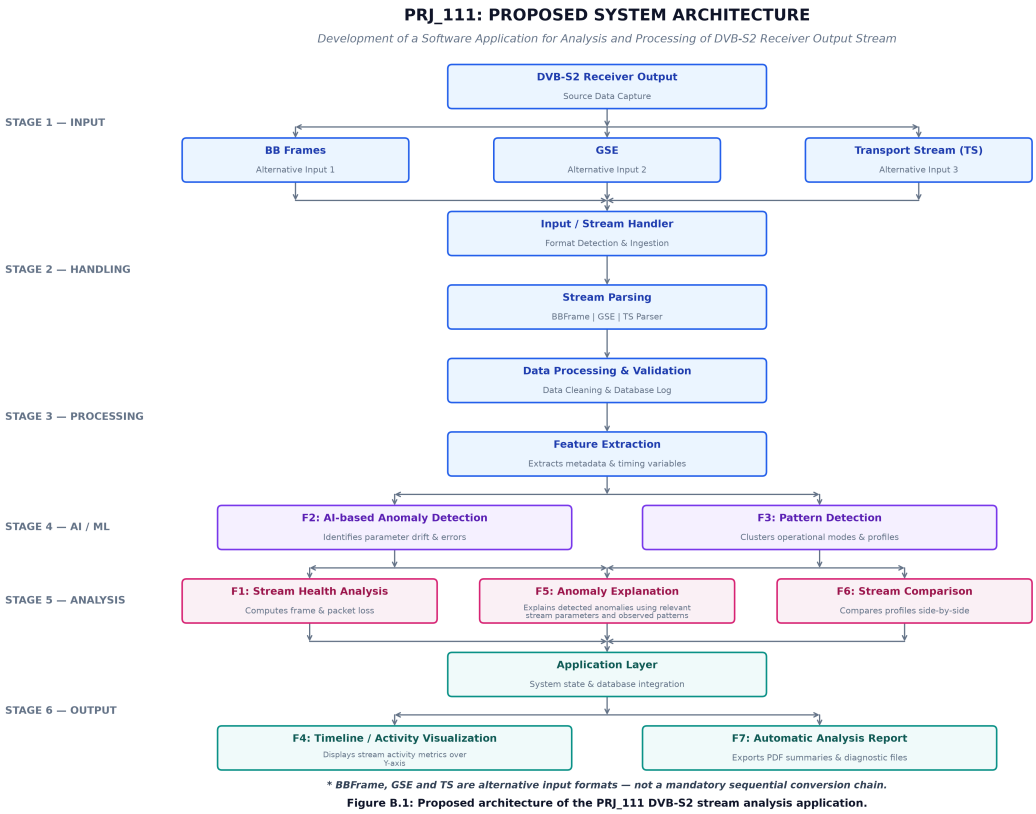


Fig. 1. End-to-End Architectural Pipeline of the PRJ_111 DVB-S2 Receiver Output Stream Analyzer (Stages 1 through 6).

B. Practical Tools and Packet Inspection

Luisi et al. [5] reviewed monitoring tools in satellite workflows, pointing out that while dedicated broadcast hardware reliably catches ETSI TR 101 290 defects, it rarely parses IP-based GSE packets or baseband frames. Meanwhile, general packet sniffers like Wireshark require Ethernet headers and struggle with raw binary stream dumps [6]. PRJ_111 fills this gap by implementing native, offline decoders for all three formats directly from raw binary bitstreams.

C. Unsupervised Anomaly Detection

Telemetry monitoring traditionally relies on fixed threshold alarms [12], [13]. These alarms trigger constantly in statistically multiplexed streams where bitrates bounce around normally. Supervised machine learning could classify anomalies, but real-world satellite ground stations rarely have labeled datasets of past outages [12].

To spot anomalies without labeled data, Liu et al. [14] developed the **Isolation Forest** algorithm. Rather than modeling what "normal" data looks like, Isolation Forest isolates unusual points by randomly slicing feature space with isolation trees (iTrees). Outlier samples sit isolated in sparse regions, meaning they require far fewer random splits to isolate and produce much shorter tree paths [14]. Baggio et al. [15] showed that Isolation Forest runs in linear time ($O(n \log n)$) and out-performs distance-based clustering algorithms like k-means++ [16] and DBSCAN [17] on high-dimensional network telemetry. We adapted Isolation Forest for DVB-S2 streams, evaluating sliding analysis windows with calibrated logistic decision scores.

III. System Architecture and Workstation

We designed PRJ_111 around a clean six-stage pipeline, illustrated in Figure 1. The software takes raw binary captures from satellite receiver outputs, decodes the container syntax, extracts normalized features, runs analytical engines (F1–F3), plots byte-accurate timelines (F4), explains detected anomalies (F5), performs semantically guarded comparisons (F6), and generates structured diagnostic reports (F7).

PRJ_111 runs on Python 3.12 as a self-contained local workstation with no external cloud services or runtime fees:

- **Core Backend and Ingestion:** Modular Python packages handle stream ingestion (`parsers/`, `features/`, `analysis/`, `streaming/`). The engine supports three ingestion modes: standard in-memory parsing, 64 KB chunked block reading for large files, and high-throughput memory-mapped (`mmap`) processing.
- **Machine Learning Layer:** Scikit-learn's `IsolationForest` runs alongside NumPy for vector calculations and clamped Z-score feature attributions.
- **Controlled Fault Injection:** A built-in synthetic mutation engine injects TEI errors, CC sequence jumps, PDU truncations, and BBFrame CRC corruptions to benchmark how sensitively F1 and F2 react to physical defects.
- **Local Web Gateway:** A multi-threaded Python HTTP server (`ThreadingHTTPServer`) serves both the REST API (`/api/status`, `/api/analyze`, `/api/compare`, `/api/upload`, `/api/export`, `/api/fault-inject`) and the frontend dashboard.
- **Operator Dashboard:** Built with vanilla HTML5, CSS3, and modern JavaScript, the single-page application gives operators byte-accurate timeline rulers, interactive window stepping, comparison delta badges, and an automated 1-click review demo.

IV. Multi-Format Stream Processing

Our parsers read raw binary captures directly, without requiring external conversion utilities:

- **MPEG-2 Transport Stream (MPEG-TS):** The decoder steps through 188-byte packets defined by ISO/IEC 13818-1 [3]. It checks the initial `0x47` sync byte, tracks the Transport Error Indicator (TEI) flag, maps 13-bit Program Identifiers (PIDs), and monitors the 4-bit cyclic Continuity Counter (CC) for missing

packets. If sync is lost, the parser scans forward byte-by-byte until cadence is restored.

- **Generic Stream Encapsulation (GSE):** Following ETSI TS 102 606-1 [4], this decoder unpacks variable-size PDUs. It reads 2-bit start/end fragmentation flags (S, E), extracts the 12-bit length field (specifying up to 4,095 bytes of payload and extension data; total PDU up to 4,097 bytes), identifies network protocols (such as IPv4 `0x0800`), verifies 6-byte or 3-byte label addressing, and validates complete PDUs with 32-bit CRC checks. In our real-world test capture, maximum observed PDU length was 1,444 bytes.
- **DVB-S2 Baseband Frames (BBFrame):** Operating under ETSI EN 302 307-1 [1], the parser validates each 80-bit header using an 8-bit CRC polynomial ($G(x) = x^8 + x^7 + x^6 + x^4 + x^2 + 1$). It decodes Mode Adaptation fields (MATYPE-1 and MATYPE-2), distinguishes single versus multiple input streams (SIS/MIS), checks modulation coding modes (CCM/ACM), and measures the Data Field Length (DFL, between 216 and 10,000 bits).

TABLE I
SUPPORTED INPUT FORMAT CHARACTERISTICS

Format	Structure	Header	Container Dimension	Checksum
MPEG-TS	Fixed Packet	4 Bytes	Fixed 188 Bytes	Continuity Counter (4b)
GSE	Variable PDU	2–8 Bytes	45–1,444 Bytes (Var.)	CRC-32 (32 bits)
BBFrame	Fixed Frame	10 Bytes	216–10,000 Bits (DFL)	CRC-8 (8 bits)

V. Feature Engineering and Analytical Pipeline

To handle different stream types fairly, we separate telemetry into universal physical metrics (`CommonMetrics`) and format-specific metrics (`TSMetrics`, `GSEMetrics`, `BBFrameMetrics`).

- **Stream Health Analysis (F1):** F1 evaluates stream quality by deducting penalties from a 100.0% clean baseline, following ETSI TR 101 290 Priority-1 rules:

$$H(W_i) = \max\Big(0.0, 100.0 + \sum \Delta H\Big) \tag{1}$$

  Windows score as `HEALTHY` ($H \geq 99.0\%$), `WARNING` ($80.0\% \leq H < 99.0\%$), or `CRITICAL` ($H < 80.0\%$).
- **AI Anomaly Detection (F2):** We configure an Isolation Forest with 100 trees (`n_estimators=100`, `contamination=0.05`). Raw anomaly decision scores $d(\boldsymbol{x})$ are mapped through a centered logistic sigmoid function:

$$s(\boldsymbol{x}) = \frac{1}{1 + e^{8.0 d(\boldsymbol{x})}} \tag{2}$$

  Windows scoring $s(\boldsymbol{x}) \geq 0.5000$ are flagged as anomalous. Because live satellite telemetry lacks verified fault labels, we evaluate model behavior through controlled synthetic mutations rather than misleading ROC-AUC scores.
- **Pattern and Entropy Analysis (F3):** F3 tracks PID allocations and computes Shannon multiplex entropy across elementary streams:

$$H_{\text{PID}} = -\sum_{k=1}^{P} p_k \log_2 p_k, \quad p_k = \frac{N_k}{\sum_{j=1}^{P} N_j} \tag{3}$$

- **Spatial Activity Timeline (F4):** Plots rolling health, anomaly scores, and byte density. Timelines are indexed strictly by physical byte offsets ($[B_{\text{start}}, B_{\text{end}}]$) and packet indices, avoiding fabricated clock timestamps.
- **Diagnostic Anomaly Explanation (F5):** Identifies which metrics triggered an anomaly flag using bounded Z-scores:

$$z_{\text{bounded}} = \text{clamp}\left(\frac{x_{i,j} - \mu_j}{\max(\sigma_j, \varepsilon_j)}, -20.0, +20.0\right) \tag{4}$$

  Clamping to $[-20.0, +20.0]$ prevents runaway division when baseline metric variance is near zero.
- **Semantic Comparison Shield (F6):** When comparing two streams of different formats, F6 strictly compares only shared physical realities (`total_payload_bytes`, `integrity_ratio`). It explicitly masks out nine format-specific counters (such as packet counts vs frame counts), preventing operators from drawing false comparisons.
- **Automatic Report Generation (F7):** Generates multi-format summaries (JSON, Markdown, HTML, text) organized under three epistemic tiers: directly observed facts, statistical findings, and engineering interpretations.

VI. Experimental Results and Discussion

We tested PRJ_111 on four real-world satellite captures stored in `01_RAW_DATA/` (Tables II and III).

TABLE II
MPEG-TS HEALTH & PRIORITY-1 INTEGRITY RESULTS

Stream	Pkts	Sync	TEI	CC	Net Payload	Health
`sample.ts` (Clean)	18,176	100%	0 (0%)	0 (0%)	3,267,305 B	100.0%
`corrupted_sample`	18,168	100%	4 (0.02%)	0 (0%)	3,265,277 B	99.67%

- **MPEG-TS Health Evaluation:** On the clean broadcast capture `sample.ts`, the parser processed 18,176 packets (3,267,305 bytes net payload) with 100.0% sync accuracy, zero TEI errors, zero continuity counter drops, and 100.0% stream health. On the corrupted stream `corrupted_sample.ts`, 8 unaligned bytes were dropped, leaving 18,168 packets. The parser caught 4 TEI-flagged packets isolated in Window 0, resulting in a 99.67% average stream health (`HEALTHY`).
- **GSE Capture Verification:** On `sample.ts` (containing GSE PDUs), the parser decoded 14 PDUs totaling 9,324 bytes (8,764 bytes payload). This capture contained 6 unfragmented PDUs (42.9%) and 8 fragmented PDUs (57.1%). All PDUs passed CRC-32 validation, scoring 100.0% health.
- **DVB-S2 Baseband Frames:** Testing on `dvb-s2_bb_example.pcap` processed 4,309 frames (2,349,376 bytes raw, 1,958,826 bytes payload). Every frame passed CRC-8 verification. Streams ran in 100% Adaptive Coding and Modulation (ACM) mode, with a dominant Data Field Length of 2,992 bits across 85.82% of frames.
- **Anomaly Detection Results:** The Isolation Forest model caught true structural outliers in each format, particularly trailing truncated windows at file boundaries (Window 90 in TS, Window 4 in GSE, Window 86 in BBFrame) and localized multiplex bursts.
- **Cross-Format Stream Comparison:** Comparing MPEG-TS against BBFrame showed a −40.05% payload difference (3,267,305 bytes vs 1,958,826 bytes, flagged `SUBSTANTIAL`) and identical 1.0000 integrity ratios (`NEGLIGIBLE`), while our semantic shield safely masked nine incommensurable counters.

TABLE III
ISOLATION FOREST ANOMALY RESULTS ACROSS FORMATS

Format	Window	Total	Anom.	Peak	Anomalous Indices
MPEG-TS	200 Pkts	91	5	0.8576	`[1, 2, 12, 88, 90]`
GSE	3 PDUs	5	1	0.5018	`[4]`
BBFrame	50 Frms	87	5	0.7406	`[0, 83, 84, 85, 86]`

VII. VERIFICATION AND LIMITATIONS

We verified the entire system using Python's standard `unittest` framework:

- **Test Results:** All 304 automated tests passed with zero failures and zero errors (268 backend unit, streaming, fault injection, and robustness tests, plus 36 frontend integration tests).
- **Operational Boundaries:** PRJ_111 operates strictly on digital bitstreams captured after demodulation. It does not measure physical radio-frequency properties like SNR or rain fade, and tracks timeline progression through byte offsets rather than assumed wall-clock times.

VIII. CONCLUSION

PRJ_111 brings MPEG-TS, GSE, and DVB-S2 Baseband Frames together into a single, offline analysis platform. By combining rule-based ETSI health checks, unsupervised Isolation Forest anomaly detection, byte-accurate timelines, clamped Z-score explanations, and a semantic comparison shield, it gives satellite engineers a trustworthy tool for inspecting ground receiver streams. Future plans include direct Software-Defined Radio (SDR) integration and FPGA-based hardware acceleration.

Acknowledgment: The authors thank Presidency University and project guide **Irfan Rajab Bhat, Assistant Professor**, for their technical guidance and computing infrastructure support.

REFERENCES

[1] "Digital Video Broadcasting (DVB); Second Generation Framing Structure, Channel Coding and Modulation Systems for Broadcasting, Interactive Services, News Gathering and Other Broadband Satellite Applications; Part 1: DVB-S2," no. ETSI EN 302 307-1 V1.4.1. Sophia Antipolis, France, 2014.

[2] A. Morello and V. Mignone, "DVB-S2: The Second Generation Standard for Satellite Broadband Services," *Proceedings of the IEEE*, vol. 94, no. 1, pp. 210–227, 2006.

[3] "Information Technology — Generic Coding of Moving Pictures and Associated Audio Information — Part 1: Systems (MPEG-2 Transport Stream)," no. ISO/IEC 13818-1:2022. Geneva, Switzerland, 2022.

[4] "Digital Video Broadcasting (DVB); Generic Stream Encapsulation (GSE); Part 1: Protocol," no. ETSI TS 102 606-1 V1.2.1. Sophia Antipolis, France, 2014.

[5] P. L. Luisi, A. B. Sampaio, and C. A. C. Marcondes, "Empirical Evaluation of Digital Video Broadcasting Systems and Transport Stream Analysis," *IEEE Transactions on Broadcasting*, vol. 66, no. 2, pp. 431–444, 2020.

[6] D. Minoli, *Innovations in Satellite Communications and the Global Information Infrastructure*. Hoboken, NJ, USA: John Wiley & Sons, 2015.

[7] E. Casini, R. De Gaudenzi, and A. Ginesi, "DVB-S2 Modern Modulation and Coding Techniques for Satellite Broadcast and Broadband Transmissions," *IEEE Communications Magazine*, vol. 42, no. 11, pp. 146–156, 2004.

[8] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2001.

[9] G. Fairhurst and B. Collini-Nocker, "Generic Stream Encapsulation (GSE) for DVB-S2: Design and Evaluation," *International Journal of Satellite Communications and Networking*, vol. 28, no. 5–6, pp. 287–305, 2010.

[10] G. Fairhurst and B. Collini-Nocker, "Unidirectional Lightweight Encapsulation (ULE) for Transmission of IP Datagrams over an MPEG-2 Transport Stream," RFC 4326, Dec. 2005.

[11] "Digital Video Broadcasting (DVB); Measurement Guidelines for DVB Systems," no. ETSI TR 101 290 V1.4.1. Sophia Antipolis, France, 2020.

[12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[13] M. A. F. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, "A Review of Novelty Detection," *Signal Processing*, vol. 99, pp. 215–249, 2014.

[14] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[15] R. S. Baggio, P. R. L. Gondim, and F. L. L. Mendonca, "Network Traffic Anomaly Detection Using Unsupervised Learning Models: A Comparative Study," *IEEE Access*, vol. 9, pp. 15982–15998, 2021.

[16] D. Arthur and S. Vassilvitskii, "k-means++: The Advantages of Careful Seeding," in *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, LA, USA, 2007, pp. 1027–1035.

[17] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, Portland, OR, USA, 1996, pp. 226–231.